

Подробный отчёт: Arabian Horse Pedigree API

Документ содержит подробное объяснение простыми словами, термины, полный актуальный код и разбор каждого пункта.

Весь код ниже — актуальная версия проекта. Файлы `node_modules` и локальный `.env` не печатаются: они содержат скачанные библиотеки и приватные настройки.

Пункт 1

Проект Node.js, Git и Express. Git хранит историю изменений; Express создаёт HTTP-сервер. GET /health возвращает 200 и JSON, то есть подтверждает работу сервера без изменения данных.

Код и объяснение: Полный код задаёт команды `npm`, исключения Git и запуск сервера.

Файл: `package.json`

```
{
  "name": "arabian-horse-pedigree-api",
  "version": "1.0.0",
  "description": "REST API for an Arabian horse pedigree database",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "create:admin": "node scripts/create-admin.js"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/mrkvqv/arabian-horse-pedigree-api.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/mrkvqv/arabian-horse-pedigree-api/issues"
  },
  "homepage": "https://github.com/mrkvqv/arabian-horse-pedigree-api#readme",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "mongoose": "^9.9.4"
  }
}
```

Полный код задаёт команды `npm`, исключения Git и запуск сервера.

Файл: `.gitignore`

```
node_modules/
.env
```

Полный код задаёт команды `npm`, исключения Git и запуск сервера.

Файл: `src/server.js`

```

const express = require('express');
require('dotenv').config({ quiet: true });

const { connectToDatabase } = require('./config/database');
const { requireAdmin } = require('./middleware/require-admin');
const countryRoutes = require('./routes/countries');
const breederRoutes = require('./routes/breeders');
const horseRoutes = require('./routes/horses');

const app = express();
const port = process.env.PORT || 3000;

app.use(express.json());

app.get('/health', (request, response) => {
  response.status(200).json({ status: 'ok' });
});

app.get('/admin/health', requireAdmin, (request, response) => {
  response.status(200).json({ status: 'ok', admin: request.admin.login });
});

```

```

app.use('/countries', countryRoutes);
app.use('/breeders', breederRoutes);
app.use('/horses', horseRoutes);

async function startServer() {
  await connectToDatabase();

  return app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
  });
}

if (require.main === module) {
  startServer().catch((error) => {
    console.error('Unable to start server:', error.message);
    process.exit(1);
  });
}

module.exports = { app, startServer };

```

Полный код задаёт команды prmt, исключения Git и запуск сервера.

Пункт 2

MongoDB работает в Docker-контейнере. Docker — изолированная готовая среда; volume сохраняет данные. .env содержит локальные настройки, а Mongoose подключается по MONGODB_URI до запуска сервера.

Код и объяснение: Код описывает контейнер, безопасный шаблон переменных и функцию подключения.

Файл: docker-compose.yml

```

services:
  mongodb:
    image: mongo:8
    container_name: arabian-horse-mongodb
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}
    volumes:
      - mongodb_data:/data/db

volumes:
  mongodb_data:

```

Код описывает контейнер, безопасный шаблон переменных и функцию подключения.

Файл: .env.example

```

MONGO_ROOT_USERNAME=admin
MONGO_ROOT_PASSWORD=change-this-password
MONGODB_URI=mongodb://admin:change-this-password@localhost:27017/arabian_horse_pedigree?authSource=admin
ADMIN_LOGIN=admin
ADMIN_PASSWORD=change-this-admin-password

```

Код описывает контейнер, безопасный шаблон переменных и функцию подключения.

Файл: src/config/database.js

```

const mongoose = require('mongoose');

// database adres from .env
async function connectToDatabase() {
  const databaseUri = process.env.MONGODB_URI;

  if (!databaseUri) {
    throw new Error('MONGODB_URI is not set. Create .env from .env.example first.');
```

Код описывает контейнер, безопасный шаблон переменных и функцию подключения.

Пункт 3

Модели Mongoose описывают документы базы: страны, заводчиков, лошадей и администраторов. Schema задаёт обязательные поля; enum ограничивает допустимые значения. ObjectId используется только внутри базы для связей.

Код и объяснение: Код содержит все схемы, базовую валидацию ссылок, индексы и правила данных.

Файл: src/models/country.js

```

const mongoose = require('mongoose');

const countrySchema = new mongoose.Schema(
  {
    code: {
      type: String,
      required: true,
      unique: true,
      uppercase: true,
      trim: true,
      match: [/^[A-Z]{2}$/, 'Country code must contain exactly two uppercase letters.'],
    },
    namePl: {
      type: String,
      required: true,
      trim: true,
    },
  },
  { timestamps: true },
);

module.exports = mongoose.model('Country', countrySchema);

```

Код содержит все схемы, базовую валидацию ссылок, индексы и правила данных.

Файл: src/models/breeder.js

```

const mongoose = require('mongoose');
const Country = require('./country');

const breederSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    country: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Country',
      required: true,
    },
    notes: {
      type: String,
      trim: true,
    },
  },
  { timestamps: true },
);

```

```

breederSchema.pre('validate', async function validateCountryReference() {
  if (!this.country) {
    return;
  }

  const countryExists = await Country.exists({ _id: this.country });

  if (!countryExists) {
    this.invalidate('country', 'Country does not exist. ');
  }
});

// A breeder is uniquely identified by its name and country of activity.
breederSchema.index({ name: 1, country: 1 }, { unique: true });

module.exports = mongoose.model('Breeder', breederSchema);

```

Код содержит все схемы, базовую валидацию ссылок, индексы и правила данных.

Файл: src/models/horse.js

```
const mongoose = require('mongoose');
const Country = require('./country');
const Breeder = require('./breeder');
```

```
const horseSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    birthYear: {
      type: Number,
      required: true,
      min: 1,
      max: new Date().getFullYear(),
    },
    sex: {
      type: String,
      required: true,
      enum: ['klacz', 'ogier', 'wałach'],
    },
  },
```

```
  coat: {
    type: String,
    required: true,
    enum: ['siwa', 'gniada', 'kasztanowata', 'kara'],
  },
  countryOfBirth: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Country',
    required: true,
  },
  mother: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Horse',
  },
  father: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Horse',
  },
  breeder: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Breeder',
    required: true,
  },
```

```
  },
  notes: {
    type: String,
    trim: true,
  },
},
{ timestamps: true },
);

//
horseSchema.pre('validate', async function validateReferences() {
  const [countryExists, breederExists, motherExists, fatherExists] = await Promise.all([
    this.countryOfBirth ? Country.exists({ _id: this.countryOfBirth }) : true,
    this.breeder ? Breeder.exists({ _id: this.breeder }) : true,
    this.mother ? this.constructor.exists({ _id: this.mother }) : true,
    this.father ? this.constructor.exists({ _id: this.father }) : true,
  ]);

  if (!countryExists) {
    this.invalidate('countryOfBirth', 'Country of birth does not exist.');
```

```

    if (!breederExists) {
      this.invalidate('breeder', 'Breeder does not exist.');
```

```
    }

    if (!motherExists) {
      this.invalidate('mother', 'Mother does not exist.');
```

```
    }

    if (!fatherExists) {
      this.invalidate('father', 'Father does not exist.');
```

```
    }
  });

  //
  horseSchema.pre('validate', async function validatePedigreeRules() {
    if (this.mother && this.father && String(this.mother) === String(this.father)) {
      this.invalidate('father', 'Mother and father must be different horses.');
```

```
    }

    const [mother, father] = await Promise.all([
      this.mother ? this.constructor.findById(this.mother).select('sex birthYear') : null,
      this.father ? this.constructor.findById(this.father).select('sex birthYear') : null,

```

```

    ]);

    if (this.mother && (await hasCurrentHorseAsAncestor(this, this.mother))) {
      this.invalidate('mother', 'Mother cannot be this horse or its descendant.');
```

```
    }

    if (this.father && (await hasCurrentHorseAsAncestor(this, this.father))) {
      this.invalidate('father', 'Father cannot be this horse or its descendant.');
```

```
    }

    validateParent(this, mother, 'mother', 'klacz');
    validateParent(this, father, 'father', 'ogier');
  });

  function validateParent(horse, parent, fieldName, expectedSex) {
    if (!parent) {
      return;
    }

    if (parent.sex !== expectedSex) {
      horse.invalidate(fieldName, `${fieldName} must reference a ${expectedSex}.`);
    }
  }

```

```

    const parentAgeAtBirth = horse.birthYear - parent.birthYear;

    if (parentAgeAtBirth < 3 || parentAgeAtBirth > 21) {
      horse.invalidate(fieldName, `${fieldName} must be between 3 and 21 years old at birth.`);
    }
  }

  async function hasCurrentHorseAsAncestor(horse, parentId) {
    const pendingIds = [parentId];
    const visitedIds = new Set();

    while (pendingIds.length > 0) {
      const currentId = pendingIds.pop();
      const currentIdString = String(currentId);

      if (currentIdString === String(horse._id)) {
        return true;
      }

      if (visitedIds.has(currentIdString)) {
        continue;
      }
    }
  }

```

```

    }

    visitedIds.add(currentIdString);

    const currentHorse = await horse.constructor
      .findById(currentId)
      .select('mother father')
      .lean();

    if (currentHorse?.mother) {
      pendingIds.push(currentHorse.mother);
    }

    if (currentHorse?.father) {
      pendingIds.push(currentHorse.father);
    }
  }

  return false;
}

// A horse is uniquely identified by its name, country of birth, and birth year

```

```

horseSchema.index({ name: 1, countryOfBirth: 1, birthYear: 1 }, { unique: true });

module.exports = mongoose.model('Horse', horseSchema);

```

Код содержит все схемы, базовую валидацию ссылок, индексы и правила данных.

Файл: src/models/admin.js

```

const mongoose = require('mongoose');

const adminSchema = new mongoose.Schema(
  {
    login: {
      type: String,
      required: true,
      unique: true,
      trim: true,
      lowercase: true,
    },
    passwordHash: {
      type: String,
      required: true,
    },
  },
  { timestamps: true },
);

module.exports = mongoose.model('Admin', adminSchema);

```

Код содержит все схемы, базовую валидацию ссылок, индексы и правила данных.

Пункт 4

API использует понятные идентификаторы: лошадь «имя код год», заводчик «название код». Они формируются из полей, а не сохраняются как технический ключ. Составные индексы защищают уникальность.

Код и объяснение: Функции проверяют код страны и формируют подписи без ObjectId.

Файл: src/utils/identifiers.js

```

function requireText(value, fieldName) {
  if (typeof value !== 'string' || value.trim() === '') {
    throw new Error(`${fieldName} must be a non-empty string.`);
  }

  return value.trim();
}

function formatCountryCode(countryCode) {
  const code = requireText(countryCode, 'countryCode').toUpperCase();

  if (!/^[A-Z]{2}$/.test(code)) {
    throw new Error('countryCode must contain exactly two letters.');
  }

  return code;
}

function formatHorseIdentifier({ name, countryCode, birthYear }) {
  if (!Number.isInteger(birthYear) || birthYear < 1) {
    throw new Error('birthYear must be a positive integer.');
```

```

  return `${requireText(name, 'name')} ${formatCountryCode(countryCode)} ${birthYear}`;
}

function formatBreederIdentifier({ name, countryCode }) {
  return `${requireText(name, 'name')} ${formatCountryCode(countryCode)} `;
}

module.exports = {
  formatHorseIdentifier,
  formatBreederIdentifier,
};

```

Функции проверяют код страны и формируют подписи без ObjectId.

Пункт 5

Родословная проверяется перед сохранением: мать — klacz, отец — ogier, возраст родителей 3–21 год, родители различны. Обход предков с Set visitedIds не допускает цикл. Hook pre validate запускается автоматически до записи в MongoDB.

Код и объяснение: В этом полном файле функции `validatePedigreeRules`, `validateParent` и `hasCurrentHorseAsAncestor` реализуют правила экзамена.

Файл: `src/models/horse.js`


```

const mongoose = require('mongoose');
const Country = require('./country');
const Breeder = require('./breeder');

const horseSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    birthYear: {
      type: Number,
      required: true,
      min: 1,
      max: new Date().getFullYear(),
    },
    sex: {
      type: String,
      required: true,
      enum: ['klacz', 'ogier', 'wałach'],
    },
  },

```

```

    coat: {
      type: String,
      required: true,
      enum: ['siwa', 'gniada', 'kasztanowata', 'kara'],
    },
    countryOfBirth: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Country',
      required: true,
    },
    mother: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Horse',
    },
    father: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Horse',
    },
    breeder: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Breeder',
      required: true,
    },
  },

```

```

  },
  notes: {
    type: String,
    trim: true,
  },
  { timestamps: true },
);

//
horseSchema.pre('validate', async function validateReferences() {
  const [countryExists, breederExists, motherExists, fatherExists] = await Promise.all([
    this.countryOfBirth ? Country.exists({ _id: this.countryOfBirth }) : true,
    this.breeder ? Breeder.exists({ _id: this.breeder }) : true,
    this.mother ? this.constructor.exists({ _id: this.mother }) : true,
    this.father ? this.constructor.exists({ _id: this.father }) : true,
  ]);

  if (!countryExists) {
    this.invalidate('countryOfBirth', 'Country of birth does not exist.');
```

```

    if (!breederExists) {
      this.invalidate('breeder', 'Breeder does not exist.');
```

```
    }

    if (!motherExists) {
      this.invalidate('mother', 'Mother does not exist.');
```

```
    }

    if (!fatherExists) {
      this.invalidate('father', 'Father does not exist.');
```

```
    }
  });

  //
  horseSchema.pre('validate', async function validatePedigreeRules() {
    if (this.mother && this.father && String(this.mother) === String(this.father)) {
      this.invalidate('father', 'Mother and father must be different horses.');
```

```
    }

    const [mother, father] = await Promise.all([
      this.mother ? this.constructor.findById(this.mother).select('sex birthYear') : null,
      this.father ? this.constructor.findById(this.father).select('sex birthYear') : null,

```

```

    ]);

    if (this.mother && (await hasCurrentHorseAsAncestor(this, this.mother))) {
      this.invalidate('mother', 'Mother cannot be this horse or its descendant.');
```

```
    }

    if (this.father && (await hasCurrentHorseAsAncestor(this, this.father))) {
      this.invalidate('father', 'Father cannot be this horse or its descendant.');
```

```
    }

    validateParent(this, mother, 'mother', 'klacz');
    validateParent(this, father, 'father', 'ogier');
  });

  function validateParent(horse, parent, fieldName, expectedSex) {
    if (!parent) {
      return;
    }

    if (parent.sex !== expectedSex) {
      horse.invalidate(fieldName, `${fieldName} must reference a ${expectedSex}.`);
    }
  }

```

```

    const parentAgeAtBirth = horse.birthYear - parent.birthYear;

    if (parentAgeAtBirth < 3 || parentAgeAtBirth > 21) {
      horse.invalidate(fieldName, `${fieldName} must be between 3 and 21 years old at birth.`);
    }
  }

  async function hasCurrentHorseAsAncestor(horse, parentId) {
    const pendingIds = [parentId];
    const visitedIds = new Set();

    while (pendingIds.length > 0) {
      const currentId = pendingIds.pop();
      const currentIdString = String(currentId);

      if (currentIdString === String(horse._id)) {
        return true;
      }

      if (visitedIds.has(currentIdString)) {
        continue;
      }
    }
  }

```

```

    }

    visitedIds.add(currentIdString);

    const currentHorse = await horse.constructor
      .findById(currentId)
      .select('mother father')
      .lean();

    if (currentHorse?.mother) {
      pendingIds.push(currentHorse.mother);
    }

    if (currentHorse?.father) {
      pendingIds.push(currentHorse.father);
    }
  }

  return false;
}

// A horse is uniquely identified by its name, country of birth, and birth year

```

```

horseSchema.index({ name: 1, countryOfBirth: 1, birthYear: 1 }, { unique: true });

module.exports = mongoose.model('Horse', horseSchema);

```

В этом полном файле функции `validatePedigreeRules`, `validateParent` и `hasCurrentHorseAsAncestor` реализуют правила экзамена.

Пункт 6

Администратор создаётся из локального `.env`. Пароль превращается `bcryptjs` в `hash` — необратимую безопасную форму. Basic Auth читает заголовок `Authorization`; `middleware` пропускает запрос или возвращает `401 Unauthorized`.

Код и объяснение: Код создаёт первого администратора и проверяет каждую защищённую операцию.

Файл: `scripts/create-admin.js`

```

require('dotenv').config({ quiet: true });

const bcrypt = require('bcryptjs');
const mongoose = require('mongoose');

const { connectToDatabase } = require('../src/config/database');
const Admin = require('../src/models/admin');

async function createAdmin() {
  const login = process.env.ADMIN_LOGIN?.trim().toLowerCase();
  const password = process.env.ADMIN_PASSWORD;

  if (!login || !password) {
    throw new Error('Set ADMIN_LOGIN and ADMIN_PASSWORD in .env first.');
  }

  await connectToDatabase();

  try {
    const existingAdmin = await Admin.findOne({ login });

    if (existingAdmin) {

```

```

        throw new Error(`Administrator "${login}" already exists.`);
    }

    const passwordHash = await bcrypt.hash(password, 12);
    await Admin.create({ login, passwordHash });
    console.log(`Administrator "${login}" was created.`);
  } finally {
    await mongoose.disconnect();
  }
}

createAdmin().catch((error) => {
  console.error(error.message);
  process.exit(1);
});

```

Код создаёт первого администратора и проверяет каждую защищённую операцию.

Файл: src/middleware/require-admin.js

```

const bcrypt = require('bcryptjs');
const Admin = require('../models/admin');

async function requireAdmin(request, response, next) {
  const authorization = request.get('authorization');

  if (!authorization?.startsWith('Basic ')) {
    return response.status(401).json({ error: 'Unauthorized' });
  }

  const [login, password] = Buffer.from(authorization.slice(6), 'base64')
    .toString()
    .split(':');

  if (!login || !password) {
    return response.status(401).json({ error: 'Unauthorized' });
  }

  const admin = await Admin.findOne({ login: login?.trim().toLowerCase() });

  if (!admin || !(await bcrypt.compare(password, admin.passwordHash))) {
    return response.status(401).json({ error: 'Unauthorized' });
  }

  request.admin = admin;
  return next();
}

module.exports = { requireAdmin };

```

Код создаёт первого администратора и проверяет каждую защищённую операцию.

Пункт 7

CRUD стран означает Create, Read, Update, Delete. GET стран публичен; POST, PUT и DELETE требуют администратора. Ответы скрывают `_id`. Страна не удаляется, если на неё уже ссылаются заводчики или лошади.

Код и объяснение: Код использует ISO-код в URL и HTTP-статусы 201, 204, 404, 409.

Файл: src/routes/countries.js

```

const express = require('express');

const Country = require('../models/country');
const Breeder = require('../models/breeder');
const Horse = require('../models/horse');
const { requireAdmin } = require('../middleware/require-admin');

const router = express.Router();

function toCountryResponse(country) {
  return {
    code: country.code,
    namePl: country.namePl,
  };
}

function getCode(code) {
  return code.trim().toUpperCase();
}

function sendDatabaseError(response, error) {
  if (error.name === 'ValidationError') {

```

```

    return response.status(400).json({ error: error.message });
  }

  if (error.code === 11000) {
    return response.status(409).json({ error: 'A country with this code already exists.' });
  }

  return response.status(500).json({ error: 'Internal server error.' });
}

router.post('/', requireAdmin, async (request, response) => {
  try {
    const country = await Country.create({
      code: request.body.code,
      namePl: request.body.namePl,
    });

    return response.status(201).json(toCountryResponse(country));
  } catch (error) {
    return sendDatabaseError(response, error);
  }
});

```

```

router.get('/', async (request, response) => {
  const countries = await Country.find().sort({ code: 1 });
  return response.json(countries.map(toCountryResponse));
});

router.get('/:code', async (request, response) => {
  const country = await Country.findOne({ code: getCode(request.params.code) });

  if (!country) {
    return response.status(404).json({ error: 'Country not found.' });
  }

  return response.json(toCountryResponse(country));
});

router.put('/:code', requireAdmin, async (request, response) => {
  try {
    const country = await Country.findOneAndUpdate(
      { code: getCode(request.params.code) },
      {
        code: request.body.code,

```

```

        namePl: request.body.namePl,
      },
      { returnDocument: 'after', runValidators: true },
    );

    if (!country) {
      return response.status(404).json({ error: 'Country not found.' });
    }

    return response.json(toCountryResponse(country));
  } catch (error) {
    return sendDatabaseError(response, error);
  }
});

router.delete('/:code', requireAdmin, async (request, response) => {
  const country = await Country.findOne({ code: getCode(request.params.code) });

  if (!country) {
    return response.status(404).json({ error: 'Country not found.' });
  }
});

```

```

const [hasBreeders, hasHorses] = await Promise.all([
  Breeder.exists({ country: country._id }),
  Horse.exists({ countryOfBirth: country._id }),
]);

if (hasBreeders || hasHorses) {
  return response.status(409).json({ error: 'Country is used by breeders or horses.' });
}

await country.deleteOne();
return response.status(204).send();
});

module.exports = router;

```

Код использует ISO-код в URL и HTTP-статусы 201, 204, 404, 409.

Пункт 8

CRUD заводчиков использует название и countryCode. При одинаковых названиях API требует countryCode и возвращает 409 при неоднозначности. populate подгружает связанную страну для ответа без ObjectId.

Код и объяснение: Код ищет, создаёт, меняет и безопасно удаляет заводчиков.

Файл: src/routes/breeders.js

```

const express = require('express');

const Breeder = require('../models/breeder');
const Country = require('../models/country');
const Horse = require('../models/horse');
const { requireAdmin } = require('../middleware/require-admin');
const { formatBreederIdentifier } = require('../utils/identifiers');

const router = express.Router();

router.use(requireAdmin);

function getCountryCode(code) {
  return code.trim().toUpperCase();
}

function toBreederResponse(breeder) {
  return {
    identifier: formatBreederIdentifier({
      name: breeder.name,
      countryCode: breeder.country.code,
    }),
  },

```

```

    name: breeder.name,
    countryCode: breeder.country.code,
    notes: breeder.notes,
  });
}

function sendDatabaseError(response, error) {
  if (error.name === 'ValidationError') {
    return response.status(400).json({ error: error.message });
  }

  if (error.code === 11000) {
    return response.status(409).json({ error: 'A breeder with this name and country already exists.' });
  }

  return response.status(500).json({ error: 'Internal server error.' });
}

async function findBreeder(name, countryCode) {
  const filter = { name: name.trim() };

  if (countryCode) {

```

```

    const country = await Country.findOne({ code: getCountryCode(countryCode) });

    if (!country) {
      return { breeder: null };
    }

    filter.country = country._id;
  }

  const breeders = await Breeder.find(filter).populate('country', 'code');

  if (!countryCode && breeders.length > 1) {
    return { ambiguous: true };
  }

  return { breeder: breeders[0] || null };
}

function sendBreederLookupError(response, lookup) {
  if (lookup.ambiguous) {
    return response.status(409).json({ error: 'Breeder name is ambiguous. Add countryCode.' });
  }
}

```

```

  if (!lookup.breeder) {
    return response.status(404).json({ error: 'Breeder not found.' });
  }

  return null;
}

router.post('/', async (request, response) => {
  try {
    const country = await Country.findOne({ code: getCountryCode(request.body.countryCode) });

    if (!country) {
      return response.status(404).json({ error: 'Country not found.' });
    }

    const breeder = await Breeder.create({
      name: request.body.name,
      country: country._id,
      notes: request.body.notes,
    });
  }
}

```

```

    await breeder.populate('country', 'code');
    return response.status(201).json(toBreederResponse(breeder));
  } catch (error) {
    return sendDatabaseError(response, error);
  }
});

router.get('/', async (request, response) => {
  const breeders = await Breeder.find().populate('country', 'code').sort({ name: 1 });
  return response.json(breeders.map(toBreederResponse));
});

router.get('/:name', async (request, response) => {
  const lookup = await findBreeder(request.params.name, request.query.countryCode);
  const errorResponse = sendBreederLookupError(response, lookup);

  if (errorResponse) {
    return errorResponse;
  }

  return response.json(toBreederResponse(lookup.breeder));
});

```

```

router.put('/:name', async (request, response) => {
  try {
    const lookup = await findBreeder(request.params.name, request.query.countryCode);
    const errorResponse = sendBreederLookupError(response, lookup);

    if (errorResponse) {
      return errorResponse;
    }

    const country = await Country.findOne({ code: getCountryCode(request.body.countryCode) });

    if (!country) {
      return response.status(404).json({ error: 'Country not found.' });
    }

    lookup.breeder.name = request.body.name;
    lookup.breeder.country = country._id;
    lookup.breeder.notes = request.body.notes;
    await lookup.breeder.save();
    await lookup.breeder.populate('country', 'code');

```

```

    return response.json(toBreederResponse(lookup.breeder));
  } catch (error) {
    return sendDatabaseError(response, error);
  }
});

router.delete('/:name', async (request, response) => {
  const lookup = await findBreeder(request.params.name, request.query.countryCode);
  const errorResponse = sendBreederLookupError(response, lookup);

  if (errorResponse) {
    return errorResponse;
  }

  const hasHorses = await Horse.exists({ breeder: lookup.breeder._id });

  if (hasHorses) {
    return response.status(409).json({ error: 'Breeder is used by horses.' });
  }

  await lookup.breeder.deleteOne();
  return response.status(204).send();

```

```

});

module.exports = router;

```


Код ищет, создаёт, меняет и безопасно удаляет заводчиков.

Пункт 9

CRUD лошадей принимает вложенные понятные ссылки на страну, заводчика и родителей. Сервер сам находит внутренние ID, применяет валидацию и возвращает пользовательские идентификаторы.

Лошадь-родителя удалить нельзя.

Код и объяснение: Код разрешает ссылки, защищает целостность дерева и подключает все маршруты к Express.

Файл: `src/routes/horses.js`

```
const express = require('express');

const Horse = require('../models/horse');
const Country = require('../models/country');
const Breeder = require('../models/breeder');
const { requireAdmin } = require('../middleware/require-admin');
const { formatHorseIdentifier, formatBreederIdentifier } = require('../utils/identifiers');

const router = express.Router();
const population = [
  { path: 'countryOfBirth', select: 'code' },
  { path: 'breeder', select: 'name country', populate: { path: 'country', select: 'code' } },
  { path: 'mother', select: 'name birthYear countryOfBirth', populate: { path: 'countryOfBirth', select: 'code' } },
  { path: 'father', select: 'name birthYear countryOfBirth', populate: { path: 'countryOfBirth', select: 'code' } },
];

router.use(requireAdmin);

function problem(status, error) {
  return { status, error };
}
```

```
function countryCode(value) {
  return typeof value === 'string' ? value.trim().toUpperCase() : '';
}

function horseReference(horse) {
  if (!horse) return null;
  const code = horse.countryOfBirth.code;
  return {
    identifier: formatHorseIdentifier({ name: horse.name, countryCode: code, birthYear: horse.birthYear }),
    name: horse.name,
    birthYear: horse.birthYear,
    countryCode: code,
  };
}

function toHorseResponse(horse) {
  const code = horse.countryOfBirth.code;
  const breederCode = horse.breeder.country.code;
  return {
    identifier: formatHorseIdentifier({ name: horse.name, countryCode: code, birthYear: horse.birthYear }),
    name: horse.name,
    birthYear: horse.birthYear,
```

```

    sex: horse.sex,
    coat: horse.coat,
    countryCode: code,
    breeder: {
      identifier: formatBreederIdentifier({ name: horse.breeder.name, countryCode: breederCode }),
      name: horse.breeder.name,
      countryCode: breederCode,
    },
    mother: horseReference(horse.mother),
    father: horseReference(horse.father),
    notes: horse.notes,
  };
}

function sendProblem(response, value) {
  return response.status(value.status).json({ error: value.error });
}

function sendDatabaseError(response, error) {
  if (error.name === 'ValidationError') return response.status(400).json({ error: error.message });
  if (error.code === 11000) return response.status(409).json({ error: 'A horse with this identifier already exists.' });
  return response.status(500).json({ error: 'Internal server error.' });
}

```

```

}

async function findCountry(code) {
  if (!countryCode(code)) return problem(400, 'countryCode is required.');
```

```

  const country = await Country.findOne({ code: countryCode(code) });
  return country || problem(404, 'Country not found.');
```

```

}

async function findBreeder(reference) {
  if (!reference?.name) return problem(400, 'breeder.name is required.');
```

```

  const filter = { name: reference.name.trim() };
  if (reference.countryCode) {
    const country = await findCountry(reference.countryCode);
    if (country.status) return country;
    filter.country = country._id;
  }
  const breeders = await Breeder.find(filter).populate('country', 'code');
```

```

  if (breeders.length === 0) return problem(404, 'Breeder not found.');
```

```

  if (breeders.length > 1) return problem(409, 'Breeder name is ambiguous. Add countryCode.');
```

```

  return breeders[0];
}

```

```

async function findHorse(reference) {
  if (!reference?.name) return problem(400, 'Horse name is required.');
```

```

  const filter = { name: reference.name.trim() };
  if (reference.birthYear !== undefined) {
    const year = Number(reference.birthYear);
    if (!Number.isInteger(year)) return problem(400, 'birthYear must be an integer.');
```

```

    filter.birthYear = year;
  }
  if (reference.countryCode) {
    const country = await findCountry(reference.countryCode);
    if (country.status) return country;
    filter.countryOfBirth = country._id;
  }
  const horses = await Horse.find(filter).populate('population');
```

```

  if (horses.length === 0) return problem(404, 'Horse not found.');
```

```

  if (horses.length > 1) return problem(409, 'Horse reference is ambiguous. Add birthYear or countryCode.');
```

```

  return horses[0];
}

async function resolveHorseData(body) {
  const [birthCountry, breeder, mother, father] = await Promise.all([
    findCountry(body.countryCode),

```

```

    findBreeder(body.breeder),
    body.mother ? findHorse(body.mother) : null,
    body.father ? findHorse(body.father) : null,
  ]);
  for (const value of [birthCountry, breeder, mother, father]) {
    if (value?.status) return value;
  }
  return { birthCountry, breeder, mother, father };
}

async function populateHorse(horse) {
  return horse.populate(population);
}

```

```

router.post('/', async (request, response) => {
  try {
    const data = await resolveHorseData(request.body);
    if (data.status) return sendProblem(response, data);
    const horse = await Horse.create({
      name: request.body.name, birthYear: request.body.birthYear, sex: request.body.sex,
      coat: request.body.coat, countryOfBirth: data.birthCountry._id, breeder: data.breeder._id,
      mother: data.mother?._id, father: data.father?._id, notes: request.body.notes,
    });
    await populateHorse(horse);
    return response.status(201).json(toHorseResponse(horse));
  } catch (error) { return sendDatabaseError(response, error); }
});

router.get('/', async (request, response) => {
  const horses = await Horse.find().populate(population).sort({ name: 1, birthYear: 1 });
  return response.json(horses.map(toHorseResponse));
});

router.get('/:name', async (request, response) => {
  const horse = await findHorse({ name: request.params.name, birthYear: request.query.birthYear, countryCode:
request.query.countryCode });
  if (horse.status) return sendProblem(response, horse);
  return response.json(toHorseResponse(horse));
});

router.put('/:name', async (request, response) => {
  try {
    const oldHorse = await findHorse({ name: request.params.name, birthYear: request.query.birthYear,
countryCode: request.query.countryCode });
    if (oldHorse.status) return sendProblem(response, oldHorse);
    const data = await resolveHorseData(request.body);

```

```

    if (data.status) return sendProblem(response, data);
    Object.assign(oldHorse, {
      name: request.body.name, birthYear: request.body.birthYear, sex: request.body.sex, coat:
request.body.coat,
      countryOfBirth: data.birthCountry._id, breeder: data.breeder._id, mother: data.mother?._id,
      father: data.father?._id, notes: request.body.notes,
    });
    await oldHorse.save();
    await populateHorse(oldHorse);
    return response.json(toHorseResponse(oldHorse));
  } catch (error) { return sendDatabaseError(response, error); }
});

router.delete('/:name', async (request, response) => {
  const horse = await findHorse({ name: request.params.name, birthYear: request.query.birthYear, countryCode:
request.query.countryCode });
  if (horse.status) return sendProblem(response, horse);
  const hasChildren = await Horse.exists({ $or: [{ mother: horse._id }, { father: horse._id }] });
  if (hasChildren) return response.status(409).json({ error: 'Horse is used as a parent.' });
  await horse.deleteOne();
  return response.status(204).send();
});

module.exports = router;

```

Код разрешает ссылки, защищает целостность дерева и подключает все маршруты к Express.

Файл: src/server.js

```
const express = require('express');
require('dotenv').config({ quiet: true });

const { connectToDatabase } = require('./config/database');
const { requireAdmin } = require('./middleware/require-admin');
const countryRoutes = require('./routes/countries');
const breederRoutes = require('./routes/breeders');
const horseRoutes = require('./routes/horses');

const app = express();
const port = process.env.PORT || 3000;

app.use(express.json());

app.get('/health', (request, response) => {
  response.status(200).json({ status: 'ok' });
});

app.get('/admin/health', requireAdmin, (request, response) => {
  response.status(200).json({ status: 'ok', admin: request.admin.login });
});

app.use('/countries', countryRoutes);
app.use('/breeders', breederRoutes);
app.use('/horses', horseRoutes);

async function startServer() {
  await connectToDatabase();

  return app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
  });
}

if (require.main === module) {
  startServer().catch((error) => {
    console.error('Unable to start server:', error.message);
    process.exit(1);
  });
}

module.exports = { app, startServer };
```

Код разрешает ссылки, защищает целостность дерева и подключает все маршруты к Express.